BCSE304L -Theory of Computation

Dr. SUGANTHINI. C
SCOPE

Module I

- Proof Techniques in Mathematics
- Overview of Computational Models
- Languages and Grammars
- Alphabets
- Strings
- Operations on Languages
- Overview on Automata

Proof Techniques in Mathematics

Structural Proof

Statement

- If H then C – If Hypothesis then Conclusion
- H iff C – Hypothesis if and only if Conclusion
  - If H then C – Assume H and prove C
  - If C then H – Assume C and prove H

Proof

- Deductive proof
- Contra-positive proof
- Contradiction based proof
- Induction
  - Mathematical
  - Structural

Deductive Proof

- Consists of a sequence of statements whose truth leads us from some initial statement
- Initial – Hypothesis
- Leads – Conclusion
- Each step in the proof must follow by some accepted logical principle
- **If $x \geq 4$ then $2^x \geq x^2$**
- Here, H is $x \geq 4$ and C is $2^x \geq x^2$
- Intuitive deductive proof
  - Each time x increases by one, the left hand side doubles in size.
  - However, the right side increases by the ratio $((x+1)/x)^2$.

Contra Positive and Contradiction

Contra Positive

- To prove “if H then C”, we would prove “if not C then not H”
- For example, the assertion "If it is my car, then it is red" is equivalent to "If that car is not red, then it is not mine".
- To prove "If P, Then Q" by the method of contra positive means to prove "If Not Q, Then Not P".

Contradiction

- To prove the statement “if H then C”, we need to prove
  - “H and not C implies falsehood”

Induction Proof

$$\sum_{i=1}^{0} i = 0$$

Basis

$$\sum_{i=1}^{n+1} i = \frac{(n+1)(n+2)}{2}$$

$$= \frac{n^2 + 3n + 2}{2}$$

Induction

(A)

Induction Proof

$$\sum_{i=1}^{n+1} i = \left(\sum_{i=1}^{n} i\right) + (n+1)$$

$$\left(\sum_{i=1}^{n} i\right) + (n+1)$$

$$= \frac{n(n+1)}{2} + (n+1)$$

$$= \frac{n^2+n}{2} + \frac{2n+2}{2}$$

$$= \frac{n^2+3n+2}{2}$$

Same as (A)

Structural Proof

- **Basic Step:** Show that the result holds for all elements specified in the basis step of the recursive definition to be in the set.
- **Recursive step:** Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

Example

- Max No of leaves in a binary tree of height h is 2^h
- Proof:
- $h = 0$, $2^0 = 1$. Only one node and basis is fine
- Tree construction

Overview of Computation Models

Chomsky hierarchy

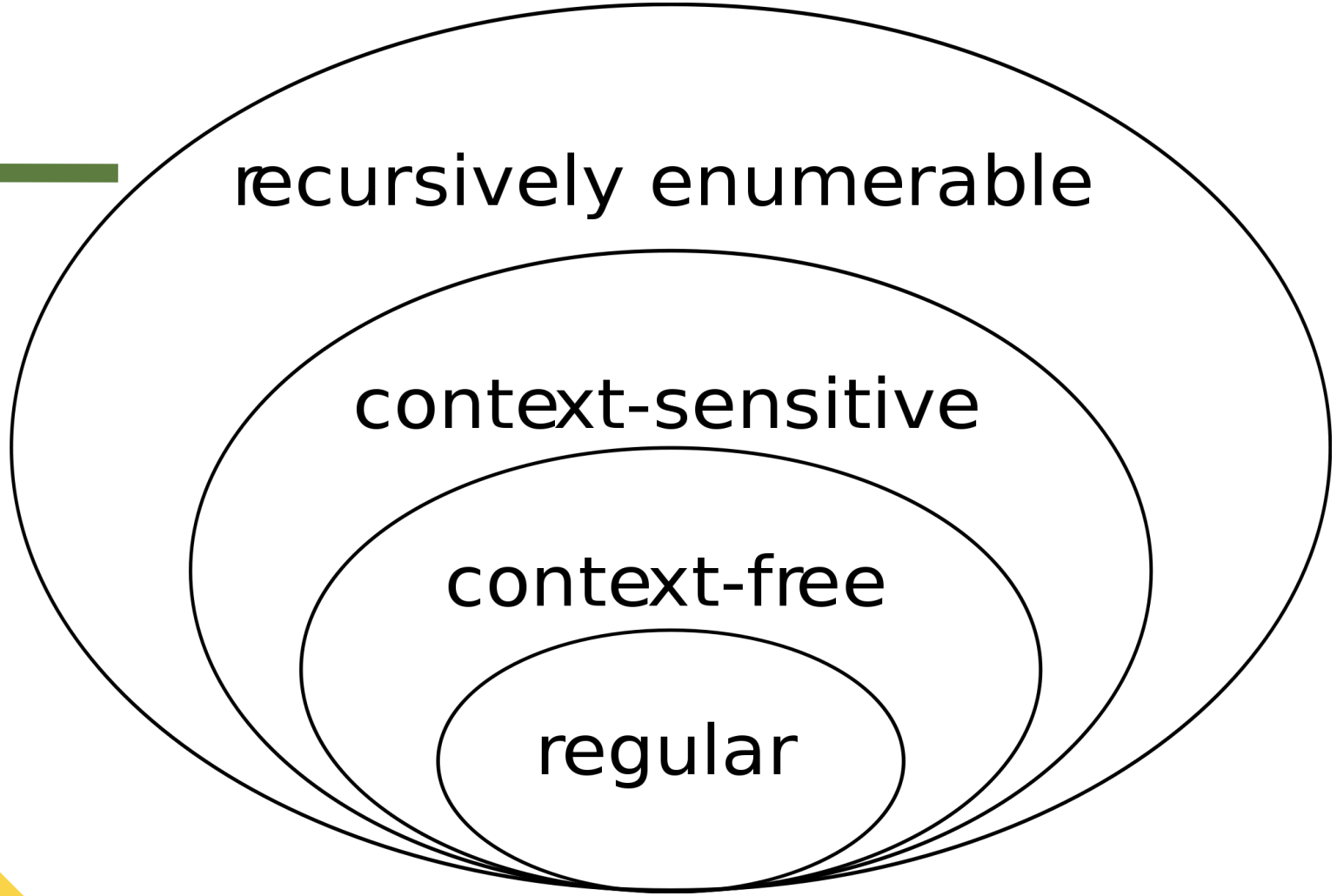
Computational Model

- Model that describes how an output of a mathematical function is computed by given an input.
  - Abstraction
  - Automation
  - Analysis

Chomsky Hierarchy

Type	Grammar	Automata	Languages	Representation
Type 0	Unrestricted Grammar	Turing Machines	Recursively Enumerable Languages	Phrase Structure
Type 1	Context Sensitive Grammar	Linear bounded Automata	Context Sensitive Languages	Context Sensitive Grammar
Type 2	Context Free Grammar	Push Down Automata	Context Free Languages	Context Free Grammar
Type 3	Regular expression	Finite State Automata	Regular Languages	Regular expression

Chomsky Hierarchy



Languages and Grammar

Regular Languages

Regular Languages

- The first language class – represented by regular expression, finite automata
- **Languages** is a set consisting of strings
- **Strings** are made up of input alphabets

Notations

- $\sum$ - {a, b} – Input alphabet
- **Alphabet** could be combined using some of the operators to get strings
- Operators typically used in the language are
  - | / + - Union operator
  - **. – Concatenation operator**
  - *** - Kleene closure operator**

Languages and Grammar

- Typical sentences are defined as

  S → NP | VP

  NP → Art Noun

  Art → A | AN | THE

  Noun → Man | Girl

  VP → Saw

- Grammar is used to check whether a sentence obeys a syntax

Languages and Grammar

- Syntax of programming languages ?
- Described by Context-Free Grammars (CFG)
  - Why named context-free?
  - Property that we can substitute strings for variables regardless of context (implies context sensitive languages exist)

Operations on Languages

Regular Expression

Regular Expression R

Basis

- **R = a** for some *a* in the alphabet $\sum$, standing for the language {a}
- R = ε, standing for the language {ε} – string of length 0
- R = Ø, standing for the empty language
- (R) where R is a regular expression, then a parenthesized R is also a regular expression

Regular Expression R

Induction

- R_1+R_2 where R_1 and R_2 are regular expressions, and + signifies union operator. '|' can also be used
- R_1R_2 where R_1 and R_2 are regular expressions and this signifies concatenation
- R* where R is a regular expression and signifies closure

Regular Expression R

- **R = a** for some *a* in the alphabet $\sum$, standing for the language {a}
- R = ε, standing for the language {ε} – string of length 0
- R = Ø, standing for the empty language
- (R) where R is a regular expression, then a parenthesized R is also a regular expression

Language Example

- R1 = a => L1 = { a }
- R2 = a + b => L2 = { a, b }
- R3 = b + a => L3 = { b , a }
- Union operator is commutative
- R4 = a.b => L4 = { ab }
- R5 = b.a => L5 = { ba }

Language Example

- R6 = a + a.b => L6 = { a , ab }
- . Has higher precedence over +
  - R7 = a . (a+b) => L7 = { aa , ab }
- Parenthesis has the highest precedence
  - R8 = a + b.(a+b) => L8 = { a, ba, bb }

Language Example

- Infinite could be generated using the * operator
- L* =

$$\bigcup_{i=0}^{\infty} L^i$$

- L* = {L^0, L^1 , L^2 ...}

Language Example

- R9 = R1* = a*

  L9 = L1* = {$L1^0$, $L1^1$, $L1^2$ ...}

- $L1^0$ = {ε}
- $L1^1$ = 1 occurrence of R1 = a
- $L1^2$ = 2 occurrences of R1 = a. a = aa
- $L1^3$ = 3 occurrences – a.a.a = aaa
  - L9 = { ε , a , aa, aaa , aaaa, ... }

Language Example

- L10 = b + a*
  - We know a* = {ε, a, aa, aaa, ...}
  - b corresponds to {b}
  - L10 = {ε , b , a, aa, aaa, ... }
  - * has the highest precedence
- L11 = (a+b)* = {ε, a , b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, bbb, ... }

Language Example

- $R\varepsilon = \varepsilon R = R$
- $(\mathbf{0+10}^*) =$

  $\{ 0, 1, 10, 100, 1000, 10000, \ldots \}$
- $(\mathbf{0^*10}^*) =$

  $\{1, 01, 10, 010, 0010, \ldots\}$

  i.e. $\{w \mid w \text{ has exactly a single } 1\}$
- $((\mathbf{0+\varepsilon)(1+\varepsilon}^*)) = \{\varepsilon, 0, 1, 01\}$ – ε is the prefix, suffix or intermediary of any string

Overview on Automata

DFA & NFA

Automata

- Directed graph that has weighted edges
- Finite automata by default means deterministic finite automata – DFA
- Non-deterministic with and without ε

Automata

- Five tuple - $(Q, \Sigma, \delta, q_0, F)$

  where Q is the finite set of states

  Σ - finite set of input symbols

  F – set of final states F is a subset of Q

  q_0 - initial state

  δ – mapping from $Q \times \Sigma$ to Q

DFA

- Design a Automata for the Switch (On & OFF).

NFA

- Design a Automata for Vending Machine.
- Let us say you have a machine that gives you water, soda and coke for Rs. 2/-, Rs. 3/- and Rs.5/- respectively. You want to represent this as a state diagram

Valid Path

11W

2W

111S

12S

21S

5C

212C

221C

122C

11111C and so on where the prefix adds up to 5followed by C

These are my valid possibilities to get water, soda, coke.

Thank You